

CS & IT ENGINEERING



Operating System

CPU Scheduling

Lecture - 07

By- Vishvadeep Gothi sir



Recap of Previous Lecture



Topic

Questions on Scheduling



Topics to be Covered



Topic

Multithreading

Topic

System Call: Fork()

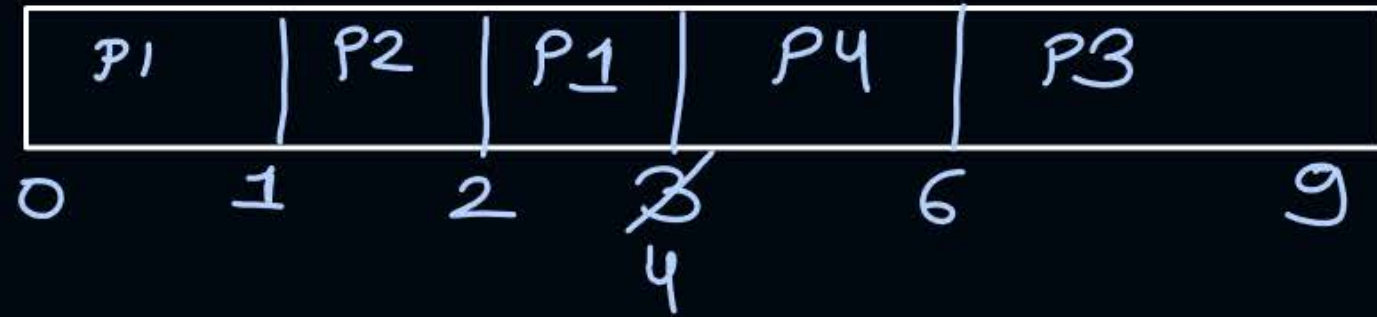
#Q. Consider the following four processes with arrival times (in milliseconds) and their length of CPU bursts (in milliseconds) as shown below:

Process	P1	P2	P3	P4
Arrival time	0	1	3	4
CPU burst Time	3 21	1	3	Z

These processes are run on a single processor using preemptive Shortest Remaining Time First scheduling algorithm. If the average waiting time of the processes is 1 millisecond, then the value of Z is 2.

$$\text{avg } WT = \frac{\text{sum of } WT}{4} = 1 \text{ ms} \Rightarrow \text{sum of } WT = 4 \text{ ms}$$

Gate - 2019



P1 (1)
P3 (3)

	WT
P1	1
P2	0
P3	1 + 2
P4	0
	4 4

Ready Queue

P3 (3)

P4 (2)

if $Z \geq 3$

P3 next

$Z < 3$

P4 next



Topic : Multithreading



Component of process

or

Lightweight Process

Provide a way to improve application performance through parallelism



Topic : Threads

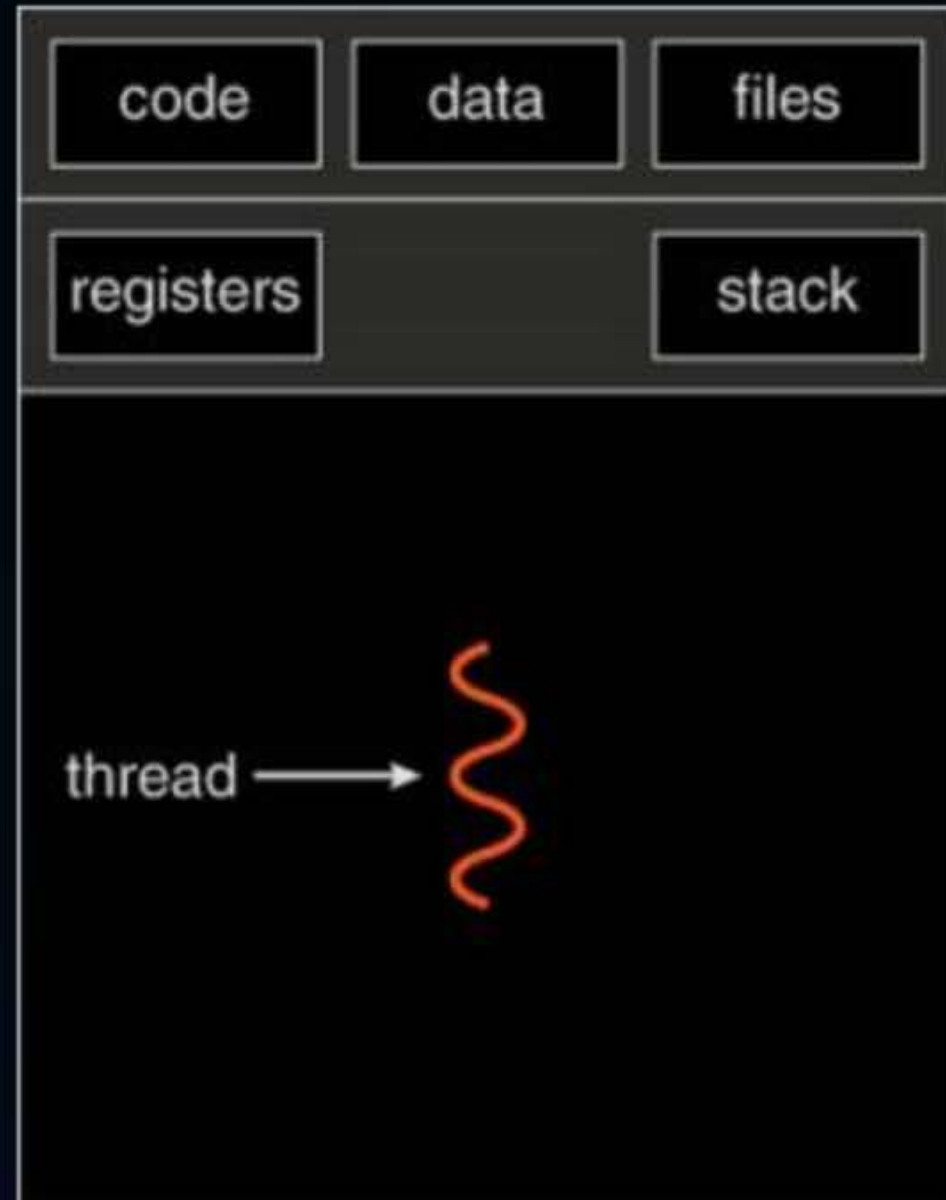


own Thread control block
(TCB)

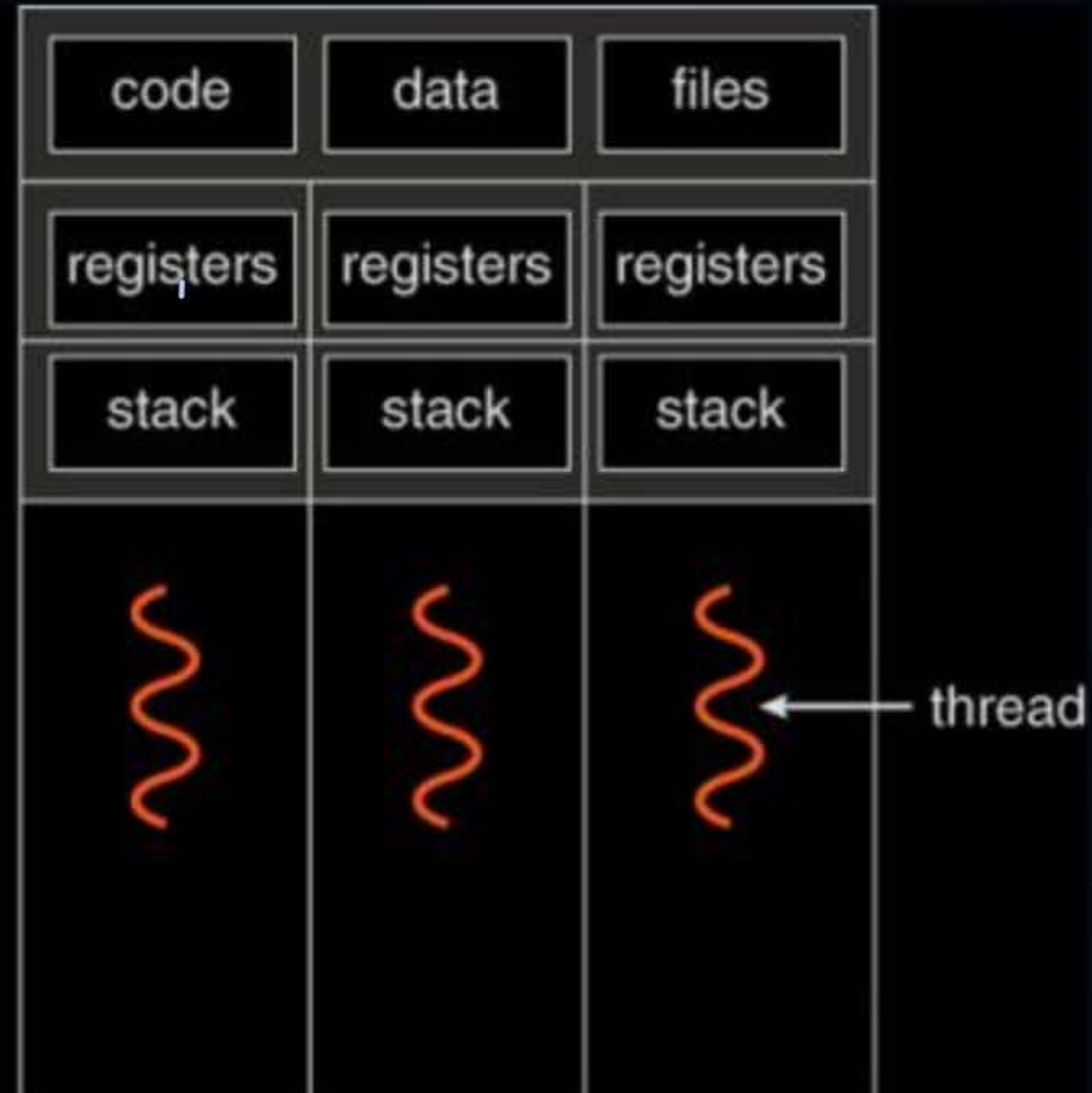
Shared Among Threads	Unique For Each Thread
Code Section	Thread Id
Data Section	Register Set
OS Resources	Stack
Open Files & Signals	Program Counter
Heap	



Topic : Threads



single-threaded process



multithreaded process



Topic : Advantages of Multithreading

- Responsiveness
- Faster Context Switch
- Resource Sharing
- Economy
- Communication
- Utilization of Multiprocessor Architecture



Topic : Types of Threads



1. User Level Thread $\Rightarrow$ user process is multithreaded
2. Kernel Level Thread $\Rightarrow$ OS ————||————



Topic : Types of Threads



User Threads	Kernel Thread
Multithreading in user process	Multithreading in kernel process
Created without kernel intervention	Kernel itself is multithreaded
Context switch is very fast	Context switch is slow
If one thread is <u>blocked</u> , OS blocks entire process <i>for I/O</i>	Individual thread can be blocked
Generic and can run on any OS	Specific to OS
Faster to create and manage	Slower to create and manage



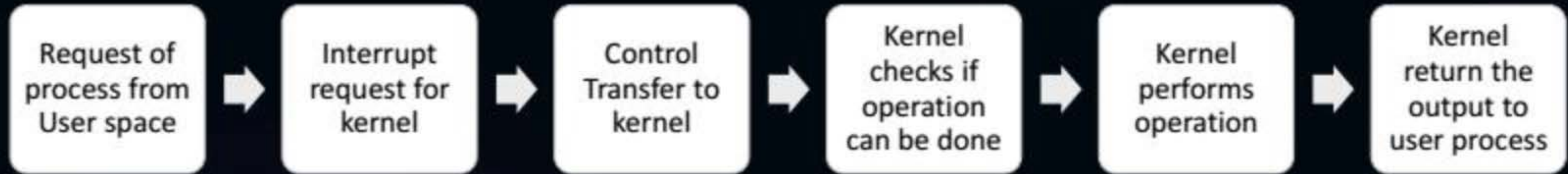
Topic : System Call



Programmatic way in which a computer program requests a service from kernel



Topic : How System Call Works



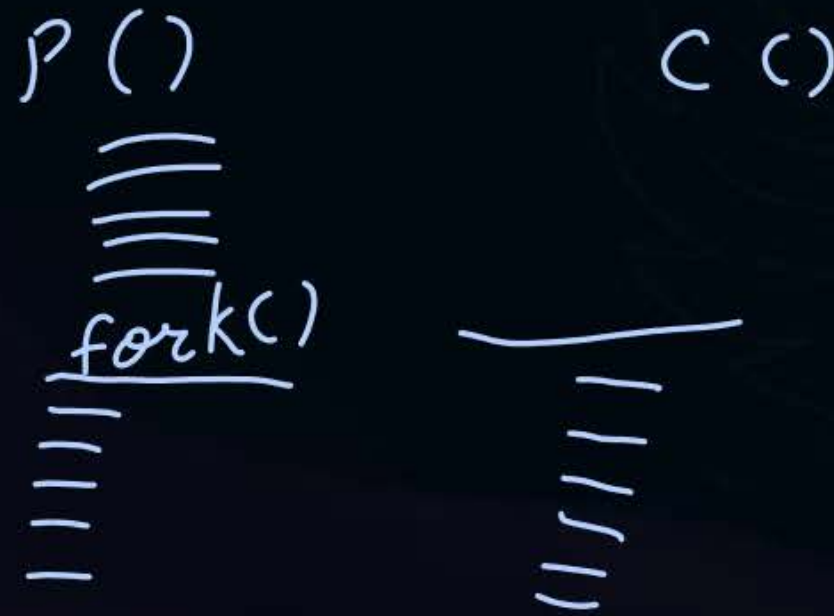


Topic : Fork() System Call

Fork system call is used for creating a new process, which is called child process.

Child process runs concurrently with the process that makes the fork() call (parent process).

child process starts execution after the fork() call which has created it.





Topic : Fork() System Call

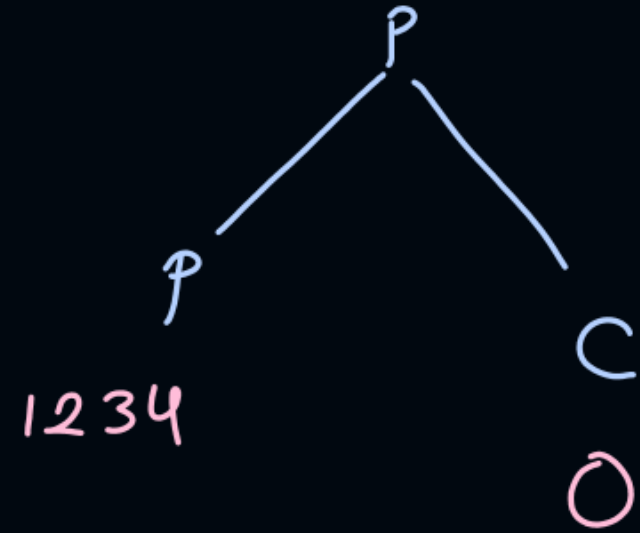
It takes no parameters and returns an integer value

- Negative Value:
Creation of a child process was unsuccessful
- Zero:
Returned to the newly created child process
- Positive value:
Returned to parent or caller. The value contains process ID of newly created child process

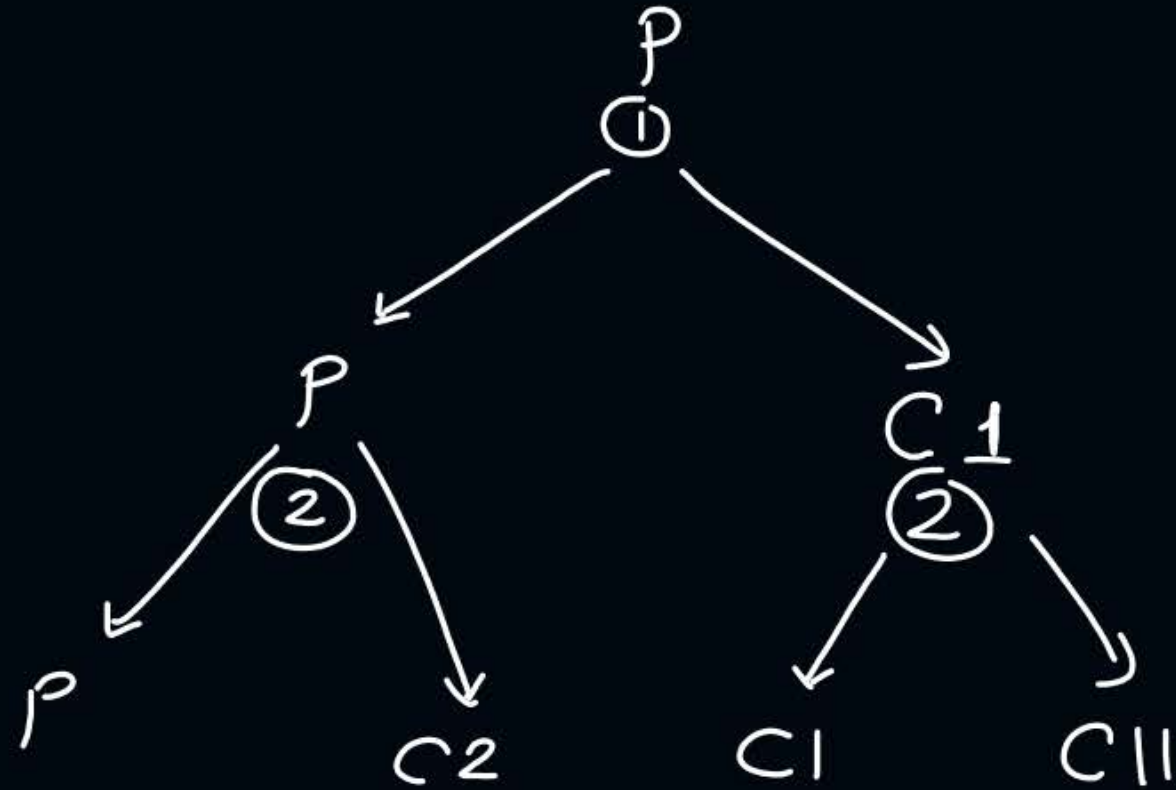
```
void main()  
{
```

```
    printf("%d", fork());  
    wait();
```

```
}
```



```
void main()
{
1.  fork();
2.  fork();
}
```



Total processes = 4 (1 parent)
3 child processes)

[MCQ]

#Q. A process executes the code

1. fork ();
2. fork ();
3. fork ();

The total number of child processes created is?

- (A) 3
(B) 4

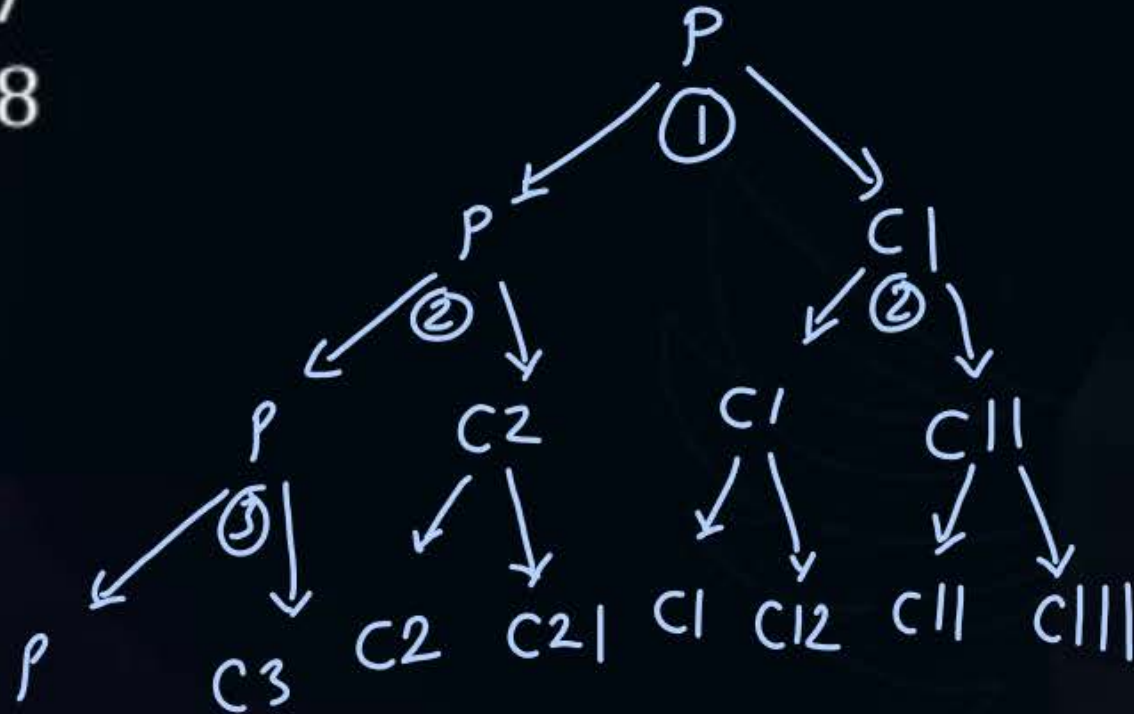
- ✓ (C) 7
(D) 8

for n consecutive fork ()

$$\text{no. of processes} = 2^n$$

$$\text{no. of child} = 2^n - 1$$

GATE-2008



Total = 8 processes

↓
1 Parent
7 child

#Q. A process executes the code
fork ();
:
fork ();

There are n such statements. The total number of child processes created is?

$$2^n - 1$$



Topic : Wait() System Call

A call to `wait()` blocks the calling process until one of its child processes completes

After child process terminates, parent continues its execution after wait system call instruction

- #Q. Consider the following code snippet using the `fork()` and `wait()` system calls. Assume that the code compiles and runs correctly, and that the system calls run successfully without any errors.

```
int x = 3;  
while(x > 0) {  
    fork();  
    printf("hello");  
    wait(NULL);  
    x--;  
}
```

The total number of times the `printf` statement is executed is _____?

GATE-2024


```
void main()
{
    if (fork() || fork())
    {
        fork();
    }
    printf("*");
}
```

no. of times '*' printed = _____ ?



2 mins Summary

Topic

Multithreading

Topic

System Call: Fork()



Happy Learning

THANK - YOU